



```
lll> cat ${CURRENT_LAB}
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title> WEB APPS </title>
```

```
</head>
```

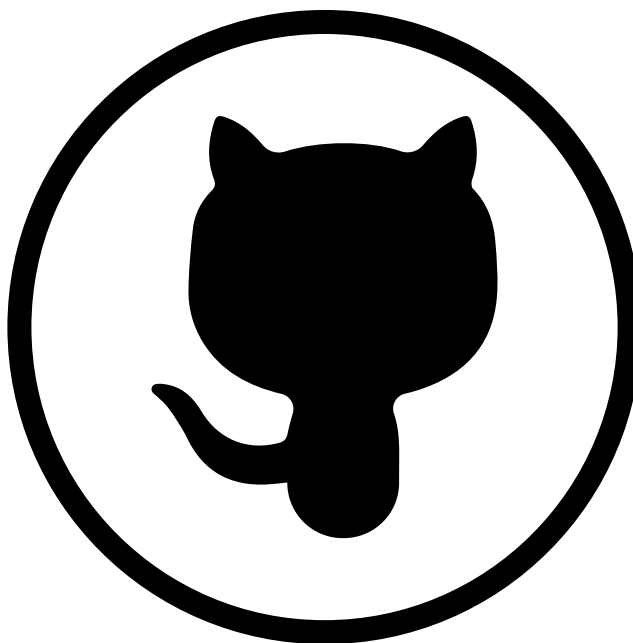
```
<body>
```

```
<h1> LAB 1 </h1>
```

```
<p> Git - Branching </p>
```

```
</body>
```

```
</html>
```



```
lll> echo "by ${AUTHOR}"
```

by Mateusz Węglewski

Task 1.0.

Przygotuj środowisko.

1. Wejdź na <https://shell.cloud.google.com> i zaloguj się kontem Google.
2. Skonfiguruj swoją tożsamość w gicie: (może wystąpić konieczność powtórzenia tego)

```
git config --global user.name "Imię Nazwisko"
git config --global user.email "twoj@email.com"
```

Task 1.1.

Sklonuj repozytorium i wejdź do środka.

```
cd ~
git clone https://github.com/eficode-academy/git-katas.git
cd git-katas
```

Task 1.2.

Wykonaj odpowiednie kroki i odpowiedz na pytania.

1. Wykonaj komendy.

```
cd basic-branching
source setup.sh
```

2. Ile aktualnie jest gałęzi? _____
3. Na której gałęzi aktualnie się znajdujesz? _____
4. Utwórz nową gałąź *mybranch* i się na nią przełącz.
5. Jak zmienia się *git status* i zawartość katalogu przy przełączaniu między *master* a *mybranch*?

6. Utwórz *file1.txt* ze swoim imieniem, dodaj i zacommituj
7. Wykonaj poniższą komendę. Jaki dała output?

```
git log --oneline --graph
```

8. Wróć na *master*, utwórz *file2.txt*, zacommituj i wykonaj poniższą komendę. Co się zmieniło?

```
git log --oneline --graph --all
```

9. Wróć na *mybranch*. Widzisz *file2.txt*? Dlaczego tak/nie?

10. Znajdź sam polecenie do porównania obu gałęzi i sprawdź różnicę. Jakie to polecenie? _____



Task 1.3.

Wykonaj odpowiednie kroki i odpowiedz na pytania.

1. Wykonaj komendy.

```
cd ~/git-katas/ff-merge
source setup.sh
```

2. Utwórz gałąź *feature/uppercase* i na nią przejdź.

3. Zmień *greeting.txt* na WERSJĘ WIELKIMI LITERAMI, dodaj, zcommituj.

4. Wykonaj poniższą komendę. Jaki dała output?

```
git log --oneline --graph --all
```

5. Wróć na *master*, porównaj gałęzie i je scal. Jakiego polecenia użyjesz do ich zmergowania?

6. Usuń gałąź *feature/uppercase* - jakim poleceniem usuwa się zmergowaną gałąź?

Task 1.4.

Na gałęzi *master* i na gałęzi *merge-conflict-branch1* ten sam plik został zmieniony inaczej. Po zakończeniu obie zmiany mają być na *master*.

1. Wykonaj komendy:

```
cd ~/git-katas/merge-conflict
source setup.sh
```

2. Wykonaj *git log --oneline --graph --all*. Ile jest gałęzi i gdzie się rozchodzą?

3. Zmerguj *merge-conflict-branch1* do *master*. Jakiego polecenia użyjesz i co się stało?

4. Co teraz pokazuje *git status*?

5. Otwórz konfliktowy plik w edytorze.

Co oznaczają znaczniki <<<<<<, ===== i >>>>>>?

6. Rozwiąż konflikt tak, żeby zostały obie zmiany, i usuń znaczniki.

7. Dokończ scalanie zgodnie z instrukcjami z *git status*. Jakich poleceń użyłeś?

8. Co teraz pokazuje *git log --oneline --graph*? Czym ten graf różni się od grafu po fast-forward z Task 1.3?



THE ENCORE

Znajdź samodzielnie (dokumentacja [git help <polecenie>](#), [git-scm.com](#)), co robi każde z wymienionych poleceń, i podaj sytuację, w której byś go użył. Chętni mogą sprawdzić je w praktyce w katach o tej samej nazwie w ~/git-katas (**basic-cherry-pick**, **basic-stashing**, **rebase-branch**, **basic-revert**, **reset**).

git cherry-pick <hash>

git stash / git stash pop

git rebase master

git revert <hash>

git reset --hard <hash>

git reflog

Zagraj w <https://learngitbranching.js.org>.

Check your knowledge

- Czym różni się **git branch** nazwa od **git switch -c nazwa**?
- Masz gałąź *main* bez żadnych nowych commitów od momentu utworzenia gałęzi *feature/x*. Jaki typ scalania wykona **git merge feature/x**?
a) merge commit b) fast-forward c) rebase d) konflikt
- Podaj angielski termin oznaczający „gałąź, która żyje krótko (godziny/dni) i jest usuwana zaraz po scaleniu”.
- Który z trzech modeli (*Git Flow / GitHub Flow / Trunk-Based Development*) zakłada osobne gałęzie *develop*, *release* / i *hotfix*?
- Zaproponuj poprawną nazwę gałęzi dla scenariusza: „*Produkcja padła, trzeba natychmiast naprawić błąd w płatnościach*.”
- Prawda czy fałsz: w modelu *GitHub Flow* gałąź *main* powinna zawsze nadawać się do wdrożenia (deployable).
- Co pokazuje polecenie **git log --oneline --graph --all**, czego nie pokaże zwykłe **git log**?
- Dlaczego w pracy zespołowej unika się nazw gałęzi typu *fix* albo *test67*?

